

מחברת 2: קריאה מרחוק (Distant Reading) 📖

מבוא למדעי הרוח הדיגיטליים | אוניברסיטת אריאל | תשפ"ו

פרופ' שי גורדין

מה נלמד במחברת זו?

קריאה מרחוק (Distant Reading) היא גישה מחקרית שבה מנתחים קורפוסים גדולים של טקסטים באמצעות כלים חישוביים, במקום לקרוא כל טקסט בנפרד (קריאה קרובה).

נושאים:

- מהי קריאה מרחוק? – תאוריה ורקע
- בניית קורפוס מוויקיפדיה העברית
- ניתוח תדירות מילים
- TF-IDF – זיהוי מילים מאפיינות
- ענני מילים (Word Clouds) בעברית
- ניתוח קונקורדנציה

קורפוס הבסיס: הטקסטים מוויקיפדיה שנאספו במחברת 1 – ארכיאולוגיה, היסטוריה, ומדעי הרוח הדיגיטליים.

חלק א: מהי קריאה מרחוק?

Franco Moretti והמהפכה הקריאה

פרנקו מורטי (Stanford) הציג את המונח "Distant Reading" ב-2000 כניגוד ל"קריאה קרובה" (Close Reading) המסורתית.

גישה	שיטה	יתרון	חסרון
קריאה קרובה	קריאת טקסטים בנפרד	עומק פרשני	מוגבל לטקסטים בודדים
קריאה מרחוק	ניתוח כמותי של קורפוסים	כיסוי רחב, דפוסים	אובדן עומק

שאלות שניתן לענות עם קריאה מרחוק:

- מהן המילים הנפוצות ביותר בתחום? (= מה ה"שפה" של השדה?)
- אילו מילים מייחדות קטגוריה אחת מאחרת?
- מהן המגמות לאורך זמן? (מתי מושגים "נולדו"?)
- מי מצוטט הכי הרבה?

לפני שמתחילים – מדריך הגדרה מהירה 🚀

שלב 1: פתחו את המחברת ב-Google Colab

לחצו על הכפתור הכחול "Open in Colab" בדף הקורס.

שלב 2: שמרו עותק אישי

File → Save a copy in Drive

(חובה – אחרת השינויים שלכם לא יישמרו!)

שלב 3: הריצו את התאים לפי הסדר – מלמעלה למטה

- **Shift + Enter** = מריץ תא נוכחי ועובר לבא אחריו
- **Ctrl + Enter** = מריץ תא נוכחי ונשאר בו
- **▶ (לחצן ירוק)** = מריץ את כל המחברת מהתחלה

זמן משוער ⌚

- שיעור: 60~ דקות (עד חלק ג')
- בית: 30~ דקות (השלמת משימת הבית)

? נתקעתם?

1. קראו את הודעת השגיאה (לרוב מסבירה מה קרה)
2. שאלו את Gemini ב-AI Studio – העתיקו את השגיאה ושאלו
3. שאלו את המרצה / הקבוצה

```
In [ ]: # התקנת ספריות
!pip install wordcloud nltk matplotlib requests tqdm -q

import nltk
nltk.download('stopwords', quiet=True)
nltk.download('punkt', quiet=True)

print("✅ הספריות הותקנו!")
print("• wordcloud – ענני מילים")
print("• nltk – עיבוד שפה טבעית בסיסי")
```

```
In [ ]: # יבוא ספריות
import requests
```

```

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
from collections import Counter
from wordcloud import WordCloud
import re
import time
import os
from tqdm.notebook import tqdm
from IPython.display import display, HTML
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

matplotlib.rcParams['axes.unicode_minus'] = False
print("✅ ספריות יובאו בהצלחה!")

```

```

In [ ]: # =====
# שליפת טקסטים מוויקיפדיה
# =====

class WikipediaTextFetcher:
    """
    שולף טקסט מלא מדפי ויקיפדיה עברית.
    משמש לבניית קורפוס לניתוח טקסט
    """

    BASE_URL = "https://he.wikipedia.org/w/api.php"

    def __init__(self, delay=0.2):
        self.session = requests.Session()
        self.delay = delay
        self.session.headers.update({
            'User-Agent': 'IntroDH-Bot/1.0 (Ariel University; Educational)'
        })

    def _request(self, params):
        """שליחת בקשה עם טיפול בשגיאות"""
        params['format'] = 'json'
        try:
            resp = self.session.get(self.BASE_URL, params=params, timeout=30)
            resp.raise_for_status()
            time.sleep(self.delay)
            return resp.json()
        except Exception as e:
            print(f"⚠️ שגיאה: {e}")
            return {}

    def get_page_text(self, title, max_chars=3000):
        """
        שליפת טקסט מלא של ערך.

        פרמטרים:
            title : כותרת הערך
            max_chars : מספר תווים מקסימלי

        מחזיר: str - הטקסט הנקי (ללא HTML)
        """

```

```

"""
data = self._request({
    'action': 'query',
    'prop': 'extracts',
    'titles': title,
    'explaintext': True,      # טקסט נקי ללא HTML
    'exlimit': 1,
    'exchars': max_chars
})
if 'query' not in data:
    return ''
for pid, pdata in data['query']['pages'].items():
    if pid != '-1':
        return pdata.get('extract', '')
return ''

def get_category_texts(self, category_name, max_pages=50, max_chars=2000):
    """
    שליפת טקסטים של כל הערכים בקטגוריה.

    פרמטרים:
        category_name : שם הקטגוריה
        max_pages      : מספר ערכים מקסימלי
        max_chars      : אורך טקסט מקסימלי לכל ערך

    רשימת dict עם title, text, category מחזיר: רשימת
    """
    # שלב 1: שליפת רשימת הערכים
    data = self._request({
        'action': 'query',
        'list': 'categorymembers',
        'cmtitle': f'קטגוריה:{category_name}',
        'cmlimit': max_pages,
        'cmtype': 'page'
    })
    if 'query' not in data:
        return []

    members = data['query']['categorymembers']
    results = []

    # שלב 2: שליפת טקסט לכל ערך
    print(f"ערכים {len(members)} שולף")
    for item in tqdm(members[:max_pages], desc=f" {category_name[:30]}"):
        text = self.get_page_text(item['title'], max_chars=max_chars)
        if text:
            results.append({
                'title': item['title'],
                'text': text,
                'category': category_name,
                'pageid': item['pageid']
            })

    return results

```

```
fetcher = WikipediaTextFetcher(delay=0.2)
print("✅ WikipediaTextFetcher מוכן!")
```

חלק ב: בניית הקורפוס

מהו קורפוס?

קורפוס (Corpus) = אוסף טקסטים שנוצר לצורך מחקר.

קורפוס טוב מאפשר לנו:

- לשאול שאלות על **שפה של תחום**
- לזהות **דפוסים** שלא נראים בקריאה רגילה
- להשוות בין **תחומים שונים**

הקורפוס שנבנה:

שלוש "תת-קורפוסים" מהקטגוריות שלנו:

1. 🏺 **ארכיאולוגיה** – שפת ממצאים, אתרים, שיטות
2. 📖 **היסטוריה** – שפת אירועים, דמויות, תקופות
3. 🖥️ **DH** – שפת כלים, מתודות, פרויקטים

🕒 **שים לב:** השליפה תיקח 5-10 דקות.

```
In [ ]: # =====
# בניית הקורפוס
# =====

CATEGORIES = [
    'ארכיאולוגיה של ארץ ישראל',
    'היסטוריה של עם ישראל',
    'מדעי הרוח הדיגיטליים'
]

CATEGORY_COLORS = {
    'ארכיאולוגיה של ארץ ישראל': '#e74c3c',
    'היסטוריה של עם ישראל': '#2980b9',
    'מדעי הרוח הדיגיטליים': '#27ae60'
}

CATEGORY_ICONS = {
    'ארכיאולוגיה של ארץ ישראל': 🏺,
    'היסטוריה של עם ישראל': 📖,
    'מדעי הרוח הדיגיטליים': 🖥️
}

print("📦 בונה קורפוס טקסטים מוויקיפדיה")
print("=" * 55)

corpus = []
```

```

for cat in CATEGORIES:
    icon = CATEGORY_ICONS[cat]
    print(f"\n{icon} {cat}:")
    texts = fetcher.get_category_texts(cat, max_pages=40, max_chars=2000)
    corpus.extend(texts)
    print(f" ✓ נאספו {len(texts)} טקסטים")

df_corpus = pd.DataFrame(corpus)
print(f"\n{'='*55}")
print(f"📊 הקורפוס כולל {len(df_corpus)} טקסטים")
print(f"סה"כ תווים: {df_corpus['text'].str.len().sum():,}")
print(f"תווים ממוצע לטקסט: {df_corpus['text'].str.len().mean():.0f}")

df_corpus.to_csv('corpus.csv', index=False, encoding='utf-8-sig')
print("\n📁 הקורפוס נשמר: corpus.csv")
display(df_corpus.head(5)[['title', 'category', 'text']].assign(text=df_corp

```

חלק ג: עיבוד טקסט – ניקוי ותקנון

למה צריך לנקות טקסט?

לפני ניתוח, עלינו לנקות ולתקן את הטקסט:

1. הסרת סימני פיסוק – אינם נושאי משמעות סמנטית
2. המרה לאותיות קטנות – "ירושלים" ו-"ירושלים" = אותה מילה (באנגלית)
3. הסרת מילות עצירה (Stop Words) – מילים נפוצות ללא משמעות ("של", "עם", "את")
4. טוקניזציה – פיצול לרשימת מילים בודדות

Stop Words בעברית:

מילות עצירה עבריות: של, עם, את, אל, על, כי, כן, לא, הוא, היא, הם...

```

In [ ]: # =====
# ניקוי ועיבוד טקסט עברי
# =====

# מילות עצירה עבריות
HEBREW_STOPWORDS = set([
    'היא', 'הוא', 'לא', 'כן', 'כי', 'על', 'אל', 'את', 'עם', 'של',
    'הם', 'הן', 'אני', 'אתה', 'את', 'אנחנו', 'אתם', 'הם',
    'זה', 'זו', 'זאת', 'אלה', 'אלו', 'כל', 'כלל', 'כל', 'יש',
    'אין', 'רק', 'גם', 'אבל', 'אם', 'כאשר', 'בין', 'אחר', 'לפני',
    'אחרי', 'בתוך', 'מחוץ', 'לאחר', 'אחד', 'שתי', 'שני', 'שלושה',
    'ב', 'ו', 'ה', 'ל', 'מ', 'כ', 'ש', 'לה', 'בה', 'בו', 'לו',
    'ממנו', 'ממנה', 'להם', 'להן', 'שהוא', 'שהיא', 'שהם',
    'הוא', 'היא', 'היתה', 'היו', 'יהיה', 'תהיה',
    'כבר', 'עוד', 'רבים', 'רבות', 'מאוד', 'יותר', 'פחות',
    'ראשון', 'שני', 'שלישי', 'אחרון', 'כמו', 'כך', 'ולכן',
    'לכן', 'לפיכך', 'משום', 'בשל', 'עקב', 'כדי', 'למרות',
    'ביחד', 'לבד', 'ללא', 'בלי', 'קצת', 'הרבה', 'יחסית',
    'מספר', 'כלשהו', 'כלשהי', 'שכן', 'מכיוון', 'היות'
])

```

```

])

def clean_hebrew_text(text):
    """
    ניקוי ותקנון טקסט עברי לניתוח.

    שלבים:
    1. הסרת תווים מיוחדים ומספרים
    2. פיצול למילים
    3. הסרת מילות עצירה ומילים קצרות מדי

    מחזיר: רשימת מילים נקיות
    """
    # ספרות, וסימנים מיוחדים, HTML, URLs, הסרת
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'<[^>]+>', '', text)
    text = re.sub(r'^\u05D0-\u05EA\s', ' ', text) # ת אותיות עבריות בלבד

    # פיצול למילים
    words = text.split()

    # סינון
    clean_words = [
        w for w in words
        if len(w) >= 2 # מינימום 2 אותיות
        and w not in HEBREW_STOPWORDS
        and not w.isdigit()
    ]

    return clean_words

def get_word_frequencies(texts):
    """חישוב תדירות מילים ברשימת טקסטים"""
    all_words = []
    for text in texts:
        all_words.extend(clean_hebrew_text(text))
    return Counter(all_words)

# עיבוד הקורפוס
print("🔪 מנקה ומעבד את הקורפוס")

df_corpus['words'] = df_corpus['text'].apply(clean_hebrew_text)
df_corpus['word_count'] = df_corpus['words'].apply(len)

print(f"✅ הטקסטים עובדו")
print(f"\n📊 סטטיסטיקות")
print(f"ממוצע מילים לטקסט: {df_corpus['word_count'].mean():.0f}")
print(f"מינימום: {df_corpus['word_count'].min()}")
print(f"מקסימום: {df_corpus['word_count'].max()}")

# דוגמה - הצגת מילים נקיות
print(f"\n🔍 דוגמה - 20 מילים ראשונות מהערך '{df_corpus.iloc[0]['title']}':")
print(df_corpus.iloc[0]['words'][:20])

```

חלק ד: ניתוח תדירות מילים

מהי תדירות מילים?

Word Frequency = כמה פעמים כל מילה מופיעה בקורפוס.

זהו הכלי הבסיסי ביותר בניתוח טקסט:

- המילים הנפוצות ביותר = הנושאים/מושגים המרכזיים
- השוואה בין קטגוריות = מה מאפיין כל שדה ידע
- שינויים לאורך זמן = התפתחות של שיח

⚠ מגבלה: Zipf's Law

תדירות מילים מתפלגת לפי **חוק זיפף**: מילים נפוצות מאוד (של, עם, ב) דומינות. לכן חשוב להסיר מילות עצירה.

```
In [ ]: # =====
# ניתוח תדירות מילים לפי קטגוריה
# =====

print("📊 מנתח תדירות מילים")

# תדירות לכל קטגוריה
category_freq = {}
for cat in CATEGORIES:
    cat_texts = df_corpus[df_corpus['category'] == cat]['text'].tolist()
    category_freq[cat] = get_word_frequencies(cat_texts)

# תדירות כוללת
all_texts = df_corpus['text'].tolist()
total_freq = get_word_frequencies(all_texts)

print("✅ חישוב הושלם!")

# — גרף: 20 המילים הנפוצות ביותר לפי קטגוריה —
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for i, cat in enumerate(CATEGORIES):
    freq = category_freq[cat]
    top20 = freq.most_common(20)
    words, counts = zip(*top20)

    color = CATEGORY_COLORS[cat]
    icon = CATEGORY_ICONS[cat]
    ax = axes[i]

    bars = ax.barh(range(len(words)), counts, color=color, alpha=0.8)
    ax.set_yticks(range(len(words)))
    ax.set_yticklabels(words, fontsize=10)
    ax.invert_yaxis()
```



```

ax.set_xlabel('תדירות', fontsize=11)
ax.set_title(f"{icon} {cat.split(' של ')[0]}\nנפוצות", fontsize=11)

# הוספת מספרים
for bar, count in zip(bars, counts):
    ax.text(bar.get_width() + 0.3, bar.get_y() + bar.get_height()/2,
            str(count), va='center', fontsize=9)

plt.suptitle('20 (לאחר הסרת מילות עצירה)\nהמילים הנפוצות ביותר לפי קטגוריה',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('02_word_frequency.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"📁 02 נשמר: word_frequency.png")

```

חלק ה: TF-IDF – מילים מאפיינות

מה זה TF-IDF?

TF-IDF (Term Frequency – Inverse Document Frequency) = מדד שמוצא מילים שמייחדות מסמך מסוים מהאחרים.

הנוסחה:

$$\text{TF-IDF (מילה, מסמך)} = \text{TF (מילה, מסמך)} \times \text{IDF (מילה)}$$

- **TF** = כמה פעמים המילה מופיעה במסמך זה
- **IDF** = $\log$ (סה"כ מסמכים / מסמכים עם המילה)

פרשנות:

- **TF-IDF גבוה** = המילה נפוצה בקטגוריה זו אבל נדירה בקטגוריות אחרות → מייחדת!
- **TF-IDF נמוך** = המילה נפוצה בכל → לא מבחינה

למה TF-IDF עדיף על תדירות פשוטה?

כי מילים כמו "ישראל" נפוצות בכל הקטגוריות, אבל "חפירה" נפוצה רק בארכיאולוגיה → TF-IDF יזהה את "חפירה" כמאפיינת.

```

In [ ]: # =====
# TF-IDF – מילים מאפיינות לכל קטגוריה
# =====

print("🔍 TF-IDF מחשב...")

# הכנה: מסמך אחד לכל קטגוריה (איחוד כל הטקסטים)
category_documents = {}
for cat in CATEGORIES:
    texts = df_corpus[df_corpus['category'] == cat]['words'].tolist()

```

```

# איחוד כל המילים לטקסט אחד
all_words = []
for word_list in texts:
    all_words.extend(word_list)
category_documents[cat] = ' '.join(all_words)

# הרצת TF-IDF
documents_list = list(category_documents.values())
category_names = list(category_documents.keys())

tfidf = TfidfVectorizer(max_features=500)
tfidf_matrix = tfidf.fit_transform(documents_list)
feature_names = tfidf.get_feature_names_out()

print(f"✅ TF-IDF! {len(feature_names)} מילים בוקטריות")

# — המובילות לכל קטגוריה TF-IDF: גרף —
fig, axes = plt.subplots(1, 3, figsize=(18, 7))

tfidf_results = {}

for i, cat in enumerate(CATEGORIES):
    # לקטגוריה זו TF-IDF ציוני
    scores = tfidf_matrix[i].toarray()[0]
    word_score = list(zip(feature_names, scores))
    word_score.sort(key=lambda x: x[1], reverse=True)

    top15 = word_score[:15]
    words, vals = zip(*top15)
    tfidf_results[cat] = top15

    ax = axes[i]
    color = CATEGORY_COLORS[cat]
    icon = CATEGORY_ICONS[cat]

    bars = ax.barh(range(len(words)), vals, color=color, alpha=0.85)
    ax.set_yticks(range(len(words)))
    ax.set_yticklabels(words, fontsize=10)
    ax.invert_yaxis()
    ax.set_xlabel('TF-IDF', fontsize=11)
    ax.set_title(f"{icon} {cat.split(' של ')[0]} (TF-IDF) מילים מאפיינות",
                 fontweight='bold', y=1.02)

    for bar, val in zip(bars, vals):
        ax.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height()/2,
                f'{val:.3f}', va='center', fontsize=9)

plt.suptitle('TF-IDF מילים המאפיינות כל קטגוריה = ייחודיות לקטגוריה זו',
             fontweight='bold', y=1.02, fontsize=13)
plt.tight_layout()
plt.savefig('03_tfidf.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"📄 03 תגובה: tfidf.png")

```

חלק ו: ענני מילים (Word Clouds)

מהו ענן מילים?

ענן מילים (Word Cloud) = ויזואליזציה שבה:

- גודל כל מילה = תדירות/חשיבות שלה
- צבע = קטגוריה

⚠ הערה ביקורתית:

ענני מילים הם ויזואליזציה מושכת אבל לא אידיאלית למחקר רציני:

- קשה להשוות גדלים עין האנושית
- עלול להסתיר מידע

עם זאת, הם שימושיים להצגה ולסקירה ראשונית.

ראו: Cairo, A. (2016). *The Truthful Art*. New Riders. (על ויזואליזציה טובה)

```
In [ ]: # =====
# ענני מילים לכל קטגוריה
# =====

print("☁ יוצר ענני מילים")

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for i, cat in enumerate(CATEGORIES):
    # בניית מילון תדירות
    freq = category_freq[cat]
    color = CATEGORY_COLORS[cat]
    icon = CATEGORY_ICONS[cat]

    # WordCloud יצירת
    # תומך בעברית אם המילים כבר מפוצלות: WordCloud: הערה
    wc = WordCloud(
        width=600,
        height=400,
        background_color='white',
        max_words=80,
        colormap='Set2',
        prefer_horizontal=0.7
    ).generate_from_frequencies(dict(freq.most_common(100)))

    ax = axes[i]
    ax.imshow(wc, interpolation='bilinear')
    ax.axis('off')
    ax.set_title(f"{icon} {cat.replace(' של ', '\nשל ')}",
                 fontsize=12, fontweight='bold', color=color)

plt.suptitle('גודל = תדירות\nענני מילים לפי קטגוריה',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig('04_wordclouds.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
print("📁 04 המילים הנשמרו: wordclouds.png")
```

חלק ז: קונקורדנציה

מהי קונקורדנציה?

קונקורדנציה = כלי קלאסי בלשנות שמציג מילה בהקשרה.

פורמט (Key Word In Context): **KWIC**:

...טקסט לפני | מילה | ...טקסט אחרי...

שימושים:

- להבין כיצד מילה מושתמשת בתחום
- לחשוף קולוקציות (מילים שמופיעות ביחד)
- לבדוק האם מושג משמש כמצופה

כלי קונקורדנציה מפורסם: [AntConc](#)

```
In [ ]: # =====
# קונקורדנציה - מילה בהקשרה
# =====

def concordance(word, texts, window=8, max_results=15):
    """
    יצירת קונקורדנציה למילה נתונה.

    פרמטרים:
        word        : המילה לחיפוש
        texts        : רשימת טקסטים
        window       : כמה מילים לפני ואחרי
        max_results  : מספר תוצאות מקסימלי

    עם הקשרים: DataFrame מחזיר
    """
    results = []

    for text_info in texts:
        text = text_info['text']
        title = text_info['title']
        cat = text_info['category']

        # פיצול למילים עם שמירת מיקומים
        words_list = re.split(r'\s+', text)

        for i, w in enumerate(words_list):
            # ניקוי המילה לבדיקה
            clean_w = re.sub(r'^\u05D0-\u05EA', '', w)
```

```

        if clean_w == word:
            # חלון לפני ואחרי
            start = max(0, i - window)
            end = min(len(words_list), i + window + 1)

            left = ' '.join(words_list[start:i])
            right = ' '.join(words_list[i+1:end])

            results.append({
                'הקשר_שמאל': left[-50:], # 50 תווים אחרונים
                'מילת_מפתח': word,
                'הקשר_ימין': right[:50], # 50 תווים ראשונים
                'ערך': title,
                'קטגוריה': cat
            })

        if len(results) >= max_results:
            return pd.DataFrame(results)

    return pd.DataFrame(results)

# — דוגמה: קונקורדנציה למילה "חפירה" —
texts_list = df_corpus.to_dict('records')

print("🔍 'חפירה': קונקורדנציה למילה:")
print("=" * 70)
df_conc = concordance('חפירה', texts_list, window=6)

if len(df_conc) > 0:
    for _, row in df_conc.iterrows():
        icon = CATEGORY_ICONS.get(row['קטגוריה'], '.')
        print(f"...{row['הקשר_שמאל'][-30:]:>30} [{row['מילת_מפתח']}] {row['קטגוריה']}"
              f"{icon} {row['ערך'][:40]}")
        print()
else:
    print("המילה לא נמצאה בקורפוס. נסו מילה אחרת.")

# — פונקציה לניסויים —
print("\n💡 נסו בעצמכם:")
print("df_conc = concordance('ירושלים', texts_list)")
print("df_conc = concordance('דיגיטלי', texts_list)")

```

```

In [ ]: # =====
# שמירת הנתונים
# =====

print("📊 שומר נתונים...")

# (שהיא רשימה words ללא עמודת) שמירת קורפוס
df_save = df_corpus.drop(columns=['words'])
df_save.to_csv('corpus.csv', index=False, encoding='utf-8-sig')
print("✅ corpus.csv")

# שמירת תדירויות
freq_data = []

```

```

for cat in CATEGORIES:
    for word, count in category_freq[cat].most_common(200):
        freq_data.append({'מילה': word, 'תדירות': count, 'קטגוריה': cat})
pd.DataFrame(freq_data).to_csv('word_frequencies.csv', index=False, encoding='utf-8')
print("✅ word_frequencies.csv")

print("\n📊 2 מחברת הושלמה!")
print()
print("📖 השלבים הבאים:")
print("📷 → OCR: 3 מחברת →")

```

תרגילים

★ תרגיל 1 – בסיסי

חפשו את 10 המילים הנפוצות ביותר בקטגוריה שמעניינת אתכם.
השוו לרשימה שקיבלנו – מה תרתה?

★★ תרגיל 2 – בינוני

הוסיפו מילות עצירה לרשימה `HEBREW_STOPWORDS` ובצעו ניתוח מחדש.
אילו מילים היה כדאי להסיר? מדוע?

★★★ תרגיל 3 – מתקדם

חוק זיפף: ניתוח כיצד מתפלגת תדירות המילים בקורפוס שלכם.
הציגו גרף log-log של תדירות מול דירוג ובדקו האם חוק זיפף מתקיים.

משאבים

- [Voyant Tools](#) – ניתוח טקסט אינטראקטיבי (ללא קוד!)
- [AntConc](#) – קונקורדנציה
- [Programming Historian: Corpus Analysis](#)
- Moretti, F. (2013). *Distant Reading*. Verso Books
- Jockers, M. (2013). *Macroanalysis*. University of Illinois Press

משימת הבית

מה להגיש:

1. בחרו קורפוס – בחרו 10–20 ערכי ויקיפדיה על נושא לבחירתכם (עיר, תקופה, אישות היסטורית...)

2. **הריצו ניתוח** – הפעילו על הקורפוס שבחרתם לפחות 2 מהניתוחים שלמדנו (תדירות מילים, TF-IDF, n-grams) ...)
3. **פרשו תוצאה אחת** – כתבו פסקה קצרה: מה גיליתם? מה מפתיע? מה מגביל את השיטה?

כיצד להגיש:

1. **File → Save a copy in Drive** – ודאו שהמחברת שמורה
2. **File → Download → Download .ipynb** – הורידו את הקובץ
3. העלו ל-Moodle

עצה: אם נתקעתם – שאלו את Gemini!

פתחו טאב חדש ב-[Google AI Studio](#) ושאלו:

אני לומד/ת "קריאה מרחוק" במדעי הרוח הדיגיטליים
נתקלתי בשגיאה הזו: "[הדביקו את השגיאה כאן]"
מה המשמעות ואיך אפשר לפתור?